Milestone 2 – Review existing work

Ayelet Zaidenberg

The data set that I would like to discuss is the "Diabetes Health Indicators Dataset" in the following link:

https://www.kaggle.com/datasets/alexteboul/diabetes-health-indicators-dataset.

This set has 253K records of 21 features to predict if a patient has no diabetes (0), prediabetes (1) or diabetes (2). The data set is not balanced. A quick check shows that ~84% of the records are categorized as no diabetes, ~14% are diabetics and only less than 2% are prediabetic.

The features are binary or numeric and include some life style features (like smoking, nutrition and general health) and some medical data (like BMI, high cholesterol and previous strokes):

- Diabetes_012 – target
- HighBP - high blood pressure: yes / no
- HighChol - high cholesterol: yes / no
- CholCheck – in the past 5 years: yes /no
- BMI – numerical
- Smoker - yes / no
- Stroke – history of stroke: yes / no
- HeartDiseaseorAttack – yes / no
- PhysActivity – yes / no
- Fruits – 1 per day: yes / no
- Veggies - 1 per day: yes / no
- HvyAlcoholConsump – heavy drinking: yes / no
- AnyHealthcare – has health care: yes / no
- NoDocbcCost – needed to see a doctor but avoided because of cost: yes / no
- GenHlth – categories 1 to 5
- MentHlth – scale 0 to 30
- PhysHlth– scale 0 to 30
- DiffWalk – difficulty walking: yes / no
- Sex – male / female
- Age – 13 age categories
- Education – 6 categories
- Income – 8 categories

Other files in the data set include a sample of 70K records from the above such that 50% are not diabetic and 50% are prediabetic / diabetic (the classification is binary) and another file with all 253K original records with binary classification 0 (not diabetic) or 1 (prediabetic / diabetic).

I would like to review some work that was done on this set and I found interesting. All code is available in the following links from Kaggle.

1. Notebook by Shohanur Sobuj, available here:
   https://www.kaggle.com/code/shohanursobuj/diabetes-health-indicators-86-76-acc

   The task here is to predict 0 / 1 classes (Not diabetic, or prediabetic / diabetic)
   This code first checks for missing data in the dataset and masseurs 0 NaN values – so no information is missing.
   Standard scaling is done only on the train data, then appling the scaler to the test data separately to avoid information leakage.
   Since the classes are imbalanced (~84% in class 0, and ~16% in class 1) we start by balancing the classes. The method selected here is SMORE (Synthetic Minority Oversampling Technique) that will generate samples with class 1 (the minority class) using KNN. This method avoids duplications and will help learn the minority class better.
   After applying this algorithm, there are now same number of samples in both classes:

```
# transform the training dataset
oversample = SMOTE(random_state=33)
X_train, y_train = oversample.fit_resample(X_train, y_train)
```

```
# summarize the new class distribution of the training dataset
counter = Counter(y_train)
print(counter)
```

```
Counter({0: 174595, 1: 174595})
```

   Then they try a few different models with different hyperparameters:

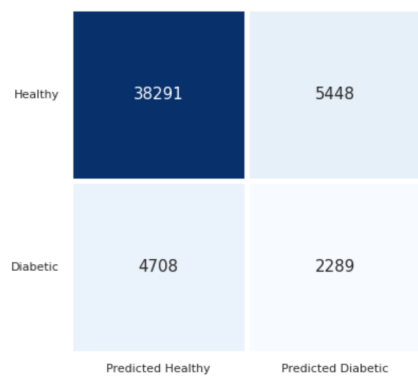
```
grid_models = [(LogisticRegression(),[{'C':[0.25,0.5,0.75,1],'ra
ndom_state':[0]}]),
               (GaussianNB(),[{'var_smoothing': [1e-09]}]),
               (DecisionTreeClassifier(),[{'criterion':['gin
i','entropy'],'random_state':[0]}]),
               (RandomForestClassifier(),[{'n_estimators':[100,1
50,200],'criterion':['gini','entropy'],'random_state':[0]}]),
               (AdaBoostClassifier(),[{'n_estimators':[100,150,2
00],'learning_rate':[0.1, 0.5, 0.8, 1],'algorithm':['SAMME', 'SA
MME.R'], 'random_state':[0]}]),
               (GradientBoostingClassifier(),[{'n_estimators':[1
00,150,200],'criterion':['friedman_mse','mse'],'loss':['devianc
e','exponential'],'learning_rate':[0.1, 0.5, 0.8, 1],'random_sta
te':[0]}]),
               (XGBClassifier(), [{'learning_rate': [0.01, 0.05,
0.1], 'eval_metric': ['error']}])]
```

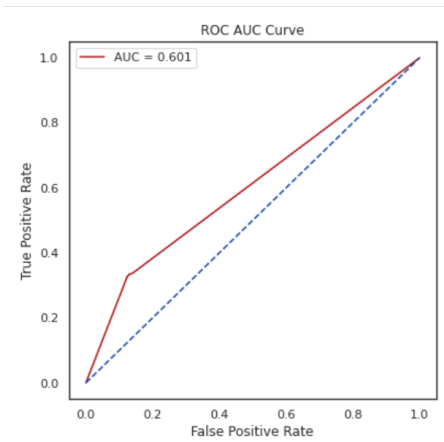
   To see what is the best tuning for each one by accuracy.

   Tuning the algorithms to the best hyperparameters, they take each algorithm and apply different measures to see how well it will detect diabetes. Here is an example for the results of a decision tree classifier with entropy as a criterion:

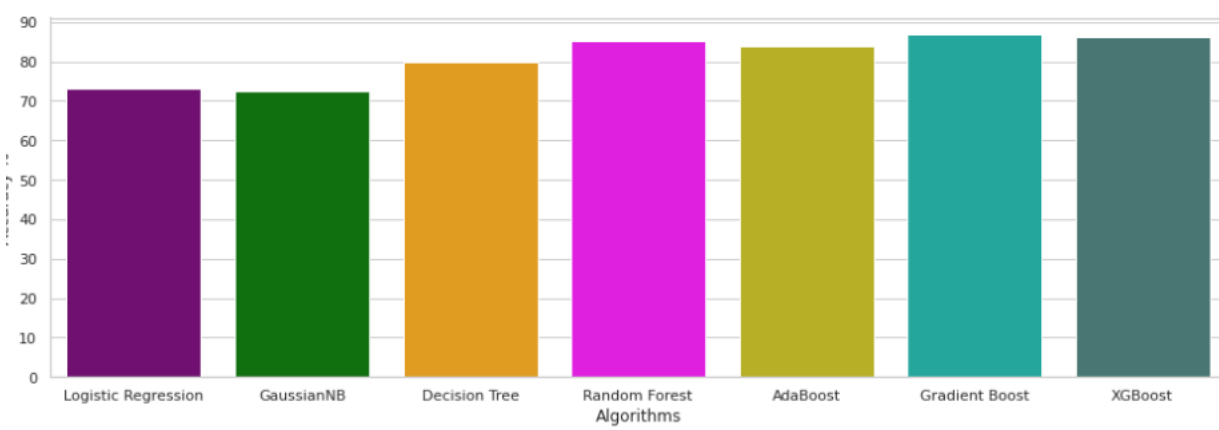
```
              precision    recall  f1-score   support

           0       0.89      0.88      0.88     43739
           1       0.30      0.33      0.31      6997

    accuracy                           0.80     50736
   macro avg       0.59      0.60      0.60     50736
weighted avg       0.81      0.80      0.80     50736

ROC AUC score: 0.6006970754055502
Accuracy Score:  0.7998265531378114
```

Confusion matrix:





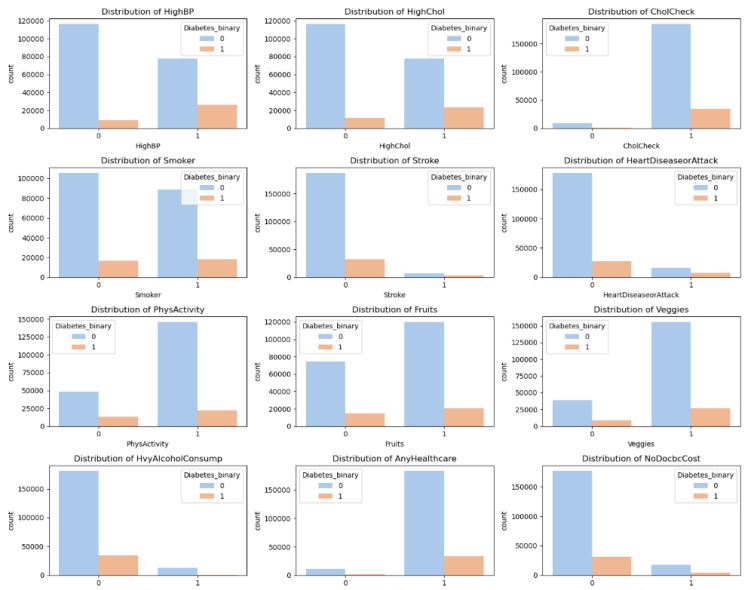
The final results were as follows:



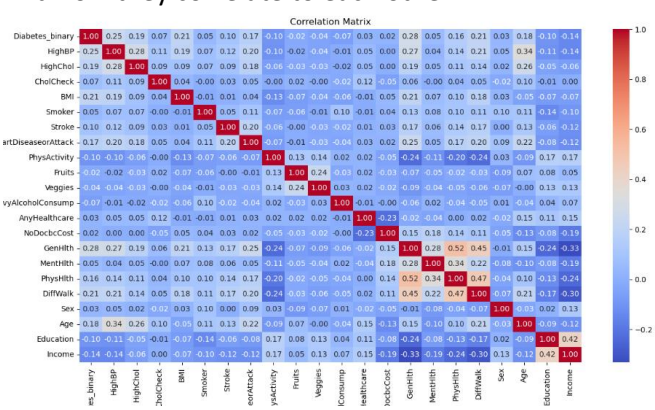
Suggesting that the boosting algorithms and Random forests are successful in this dataset (giving the adjustments for the train data).

2. Another approach by Adison Goh is to try Logistic regression.
   https://www.kaggle.com/code/adisongoh/binary-classification-of-patients-with-diabetes

   They first drop the duplicates from the set. Since only 1% of the positive data is duplicated, it is not problematic.
   Then features are separated to binary classified, numeric and ordinal and we check how they correlate with the data:



   And how they correlate to each other:



   No very strong correlations are found although some moderate correlations are found.

   Appling basic Logistic regression has poor result, however when the hyperparameter "class_weight" is set to "balanced" the performance improve since points of the positive class get higher weight. Adding squared features improve the classifier even better.

```python
# Initialise standard scaler
scaler = StandardScaler()
# Apply degree=2 to polynomial features
poly = PolynomialFeatures(degree=2, interaction_only=True)

# Create a logistic regression model without regularisation but wi
th balanced class weights, apply solver='lbfgs'
logr = LogisticRegression(solver='lbfgs', class_weight = 'balanc
ed')

# Create a pipeline with a standard scaler and the logistic regres
sion model
pipe = Pipeline([
   #(nickname, step)
    ('poly', poly),
    ('scaler', scaler),
    ('logr', logr)
])
```

Maybe due to some of the correlation between the features.

- Improvement in AUC-ROC: 0.56 to 0.73
- Balanced accuracy increased from 0.56 to 0.73
- Overall F1-score decreased from 0.81 to 0.76
- F1-score for minority class (1) increased from 0.25 to 0.45
- Recall of minority class (1) increased from 0.14 to 0.75
- F1-score for majority class (0) decreased from 0.92 to 0.81
- Recall for majority class (0) decreased from 0.98 to 0.72

Using measurements like F1-score and recall can show us that this model is better in detecting cases of positive for diabetics, even without adding syntactic data, but by changing the settings of logistic regression.

I believe we may still be able to improve the result for the binary classification problem by trying more models and settings, and other algorithms to overcome the imbalanced data, such as other algorithms of creating synthetic data.
Another problem that can be interesting is to apply some of the techniques to the trinary classification problem trying different approaches to multiclassification.